

ProSpy: Fake Social Media Account Detection Web Application

Backend Technical Documentation

Author: Archi Sharma and Parth Goyal

Department: Computer Science and Engineering

Date: July 2026

Table of Contents

1. Title Page	1
2. Table of Contents	2
3. Abstract	3
4. Introduction	4
5. Technology Stack	7
6. Backend Architecture	10
7. Project Folder Structure	14
8. Flask Application	18
9. Machine Learning Model Integration	22
10. API Documentation	27
11. Request Processing Workflow	30
12. Data Preprocessing	34
13. Error Handling	37
14. Security	40
15. Performance Optimization	43
16. Deployment	46
17. Testing	50
18. Challenges Faced	53
19. Future Enhancements	56
20. Conclusion	59
21. References	60
22. Appendix	61

3. Abstract

The backend component of ProSpy is the computational engine responsible for processing user-submitted social media profile features and delivering accurate fake account predictions powered by a deep learning model. Built with Flask, it provides a scalable, secure, and efficient RESTful API layer that integrates seamlessly with the React frontend and a TensorFlow/Keras neural network.

This comprehensive documentation details every aspect of the backend — from architecture and model integration to deployment on Render, security hardening, and performance tuning. It emphasizes production-ready practices including one-time model loading, robust preprocessing pipelines, comprehensive error handling, and adherence to REST principles.

Designed for a B.Tech final-year project, this report ensures reproducibility, maintainability, and academic rigor while highlighting the critical role of backend services in AI-driven applications. The implemented system demonstrates low-latency inference, high reliability, and privacy-conscious design by avoiding unnecessary data persistence. (Word count: ~210)

4. Introduction

4.1 Overview of the Backend System

The ProSpy backend is a Python-based web service that receives JSON payloads containing extracted social media account features, performs necessary preprocessing, runs inference on a trained neural network, and returns structured prediction results. It acts as the secure intermediary between the client interface and the machine learning logic.

4.2 Role of the Backend in the ProSpy Architecture

In the overall client-server architecture, the backend encapsulates all business and AI logic. It decouples the frontend from model specifics, allowing independent updates to the ML component without frontend changes. This separation supports horizontal scaling and easier debugging.

4.3 Communication between Frontend and Machine Learning Model

Communication occurs exclusively via HTTPS POST requests to the /predict endpoint. The frontend sends feature vectors; the backend validates, transforms, and forwards them to the loaded Keras model, returning JSON with prediction and confidence metrics. This design ensures loose coupling and technology agnosticism.

5. Technology Stack

Python: The foundational language chosen for its rich ecosystem in data science and web development, readability, and extensive library support.

Flask: A lightweight, flexible micro-framework ideal for REST APIs. It provides routing, request handling, and extensibility without unnecessary overhead.

Flask-CORS: Middleware for managing Cross-Origin Resource Sharing policies, essential for secure frontend-backend interaction during development and production.

TensorFlow/Keras: Industry-standard deep learning framework. Keras offers a high-level API for loading and executing the pre-trained classification model efficiently.

Scikit-learn: Used for data preprocessing utilities, specifically the StandardScaler fitted during training.

NumPy & Pandas: NumPy enables fast array operations; Pandas simplifies tabular data manipulation and DataFrame-based feature handling.

Joblib: Efficient serialization library for persisting and loading Python objects such as the scaler and selected feature lists.

Gunicorn: Mature WSGI server that manages multiple worker processes, enabling concurrent request handling in production environments.

Table 5.1: Dependencies and Purposes

Component	Version Range	Purpose	Rationale
Flask	3.0+	API Framework	Simplicity & extensibility
TensorFlow	2.15+	Model Serving	Optimized inference
Gunicorn	22+	Production Server	Concurrency & reliability
Joblib	1.4+	Artifact Loading	Fast binary serialization

6. Backend Architecture

Flask Application Architecture: Modular design using application factory pattern for testability and configuration flexibility.

REST API Architecture: Stateless, resource-based endpoints following RESTful conventions. JSON is the primary data format.

Request Lifecycle: Detailed flow from HTTP arrival through Gunicorn, Flask routing, middleware, business logic, to response.

Data Flow: Input JSON → Validation → DataFrame → Preprocessing Pipeline → Model Inference → Output JSON.

7. Project Folder Structure

The backend of ProSpy is organized into a modular structure that separates the application logic, machine learning model, and configuration files. This organization improves maintainability, readability, and ease of deployment.

app.py / main.py

This is the main entry point of the backend application. It initializes the Flask application, configures CORS, loads the trained machine learning model and preprocessing files, defines API routes such as `/predict` and `/health`, and starts the web server using Gunicorn during deployment.

requirements.txt

The `requirements.txt` file lists all Python dependencies required by the backend, including Flask, TensorFlow, Scikit-learn, NumPy, Pandas, Joblib, Flask-CORS, and Gunicorn. Pinning package versions ensures a reproducible environment and prevents compatibility issues during deployment.

best_model.keras

This file contains the trained neural network model saved in Keras format. It stores the model architecture, learned weights, and configuration, allowing the backend to perform predictions without retraining the model.

scaler.pkl

The `scaler.pkl` file is a Joblib-serialized **StandardScaler** object used to standardize input features before prediction. Using the same scaler as during training ensures consistent preprocessing and accurate model inference.

selected_features.pkl

This file stores the list of input features selected during model training. It ensures that prediction requests use the correct feature names and order expected by the trained model.

categorical_encoding_info.json

This JSON file contains mappings used for encoding categorical variables into numerical values. It maintains consistency between training and prediction by applying the same encoding scheme to incoming data.

Utility Files

Additional utility files may be included to improve code organization and maintainability. These files typically handle tasks such as input validation, application configuration, helper functions, logging, and data preprocessing. Separating reusable functionality into utility modules makes the backend easier to maintain, extend, and debug while promoting cleaner and more modular code.

8. Flask Application

The ProSpy backend is built using the **Flask** web framework, which provides a lightweight and efficient environment for developing RESTful APIs. The application is initialized by creating a Flask instance and enabling **Cross-Origin Resource Sharing (CORS)** to allow secure communication with the React frontend. Since the project is relatively small, the application is implemented in a single `app.py` file, and **Blueprints are not used**. However, the modular structure allows Blueprints to be added easily as the project grows.

Route decorators such as `@app.route()` define the API endpoints, including the prediction endpoint (`/predict`) and the health check endpoint (`/health`). These routes receive HTTP requests, process user input, and return JSON responses to the frontend. JSON utilities provided by Flask, such as `request.get_json()` and `jsonify()`, are used for parsing incoming data and generating structured responses.

To ensure secure communication, **Flask-CORS** is configured to allow requests only from the authorized frontend origin in the production environment while preventing unauthorized cross-origin access. Centralized error handling is implemented using `try-except` blocks and Flask's built-in error handlers to manage invalid requests, missing fields, and internal server errors gracefully. Meaningful HTTP status codes and descriptive JSON error messages are returned, improving both user experience and debugging.

The following code snippet demonstrates the basic Flask application setup:

```
from flask import Flask, request, jsonify

from flask_cors import CORS

from tensorflow.keras.models import load_model

import joblib

app = Flask(__name__)

# Enable CORS for the frontend

CORS(app, origins=["https://your-frontend-domain.com"])

# Load model and preprocessing files

model = load_model("best_model.keras")

scaler = joblib.load("scaler.pkl")
```



```
selected_features = joblib.load("selected_features.pkl")
```

Example route definition:

```
@app.route("/predict", methods=["POST"])
```

```
def predict():
```

```
    data = request.get_json()
```

```
    # Preprocess input and generate prediction
```

```
    return jsonify({"prediction": "Fake Account"})
```

Health check endpoint:

```
@app.route("/health", methods=["GET"])
```

```
def health():
```

```
    return jsonify({"status": "healthy"})
```

Overall, the Flask application provides a simple, secure, and scalable backend architecture that efficiently integrates the machine learning model with the frontend while supporting reliable API communication and maintainable code structure.

9. Machine Learning Model Integration

The ProSpy backend integrates the trained neural network model with the Flask application to provide real-time predictions. To improve performance, the model and all preprocessing objects are **loaded once during application startup** and stored as global variables. This avoids repeated disk access and model initialization for every request, significantly reducing prediction latency. A singleton-style loading approach ensures that a single instance of the model, `StandardScaler`, and selected feature list is reused throughout the application's lifetime.

Detailed Loading Logic

During startup, the backend loads the trained model and preprocessing files:

- `best_model.keras` – Trained neural network model.
- `scaler.pkl` – StandardScaler used during training.

- `selected_features.pkl` – List of input features expected by the model.

This initialization occurs only once, allowing all incoming requests to use the same in-memory objects for efficient inference.

Preprocessing Pipeline

Before prediction, the backend follows a standardized preprocessing pipeline:

1. Receive the JSON request from the frontend.
2. Validate the input fields and data types.
3. Convert the input into a Pandas DataFrame.
4. Arrange the features in the same order used during model training.
5. Apply the saved **StandardScaler** to standardize numerical values.
6. Convert the processed data into the format required by the neural network.

This pipeline ensures that prediction inputs are consistent with the data used during model training.

Inference

Once preprocessing is complete, the standardized input is passed to the neural network using the Keras `model.predict()` method. The model outputs a probability score indicating the likelihood that the account is fake. This probability is extracted from the prediction output and used for the final classification.

Prediction Generation

The backend applies a threshold-based decision rule to convert the probability into a class label. If the predicted probability is **greater than 0.5**, the account is classified as **Fake**; otherwise, it is classified as **Genuine**. The confidence score is calculated by converting the predicted probability into a percentage and is returned along with the classification result in a JSON response. This process provides users with both the prediction and an indication of the model's confidence, making the output more informative and easier to interpret.

10. API Documentation

The ProSpy backend exposes a RESTful API endpoint that allows the frontend to submit social media account details and receive a prediction indicating whether the account is genuine or fake.

Endpoint

POST /predict

Request Headers

Header	Value
Content-Type	application/json
Accept	application/json

Request Body Schema

The request body must be sent in JSON format with the required input features.

Field	Type	Description
username_length	Integer	Length of the username
follower_count	Integer	Number of followers
following_count	Integer	Number of accounts followed

post_count	Integer	Total number of posts
bio_length	Integer	Length of the profile biography
has_profile_pic	Integer (0/1)	Indicates whether a profile picture exists
is_verified	Integer (0/1)	Verification status of the account
has_external_url	Integer (0/1)	Indicates whether an external URL is present
engagement_rate	Float	User engagement rate
account_age_days	Integer	Age of the account in days

Sample JSON Request

```
{
  "username_length": 15,
  "follower_count": 2500,
  "following_count": 340,
  "post_count": 180,
  "bio_length": 120,
  "has_profile_pic": 1,
  "is_verified": 0,
  "has_external_url": 1,
```

```
"engagement_rate": 4.2,  
"account_age_days": 890  
}
```

Response Schema

The backend returns a JSON response containing the predicted class and confidence score.

Field	Type	Description
prediction	String	Predicted class (Fake Account / Genuine Account)
confidence	Float	Confidence score (%)

Sample JSON Response

```
{  
  "prediction": "Fake Account",  
  "confidence": 96.82  
}
```

HTTP Status Codes

Status Code	Description
200 OK	Prediction generated successfully.
400 Bad Request	Invalid request format, missing fields, or malformed JSON.

422 Unprocessable Entity	Input data is syntactically correct but contains invalid or unacceptable values.
500 Internal Server Error	An unexpected server-side error occurred during preprocessing or model prediction.

Error Response Example (400 Bad Request)

```
{  
  "error": "Missing required fields in the request."  
}
```

Error Response Example (422 Unprocessable Entity)

```
{  
  "error": "Invalid input values. Please verify the data types and ranges."  
}
```

Error Response Example (500 Internal Server Error)

```
{  
  "error": "An internal server error occurred while processing the prediction request."  
}
```

The `/predict` endpoint serves as the primary communication interface between the React frontend and the Flask backend. It accepts validated user input, performs preprocessing and model inference, and returns a structured JSON response containing the prediction and confidence score, ensuring reliable and efficient real-time fake account detection.

11. Request Processing Workflow

The ProSpy backend processes prediction requests through the following steps:

1. **Receive Request** – The Flask backend receives user input as a JSON request from the React frontend.
2. **Validate Input** – The input data is checked for completeness, correct data types, and valid values.
3. **Data Preprocessing** – The required features are selected and standardized using the saved `StandardScaler`.
4. **Model Prediction** – The preprocessed data is passed to the trained neural network (`best_model.keras`) to generate a prediction.
5. **Generate Response** – The prediction result and confidence score are formatted into a JSON response.
6. **Return Result** – The JSON response is sent back to the frontend, where it is displayed to the user in a readable format.

12. Data Preprocessing

Before prediction, the backend preprocesses the input data to ensure consistency with the training data. It validates all required fields, checks data types and value ranges, and aligns the input features in the same order used during model training. The selected features are then standardized using the saved **StandardScaler** to improve prediction accuracy.

If any required values are missing or invalid, the request is rejected with an appropriate error message instead of assigning default values. The entire preprocessing pipeline is encapsulated within the backend, ensuring that every prediction follows the same standardized workflow used during model development.

13. Error Handling

The ProSpy backend implements robust error handling to ensure reliable and uninterrupted operation. User inputs are validated before processing, and any invalid or incomplete requests are handled gracefully with clear and informative error messages. Flask's built-in error handling mechanisms are used to catch common exceptions, such as invalid JSON data, missing required fields, unsupported request methods, and internal server errors.

Unexpected exceptions during preprocessing or model prediction are captured using `try-except` blocks, preventing the application from crashing. Appropriate HTTP status codes, such as **400 (Bad Request)** for invalid input and **500 (Internal Server Error)** for server-side issues, are returned to the client. Error details are logged to assist in debugging and monitoring while avoiding the exposure of sensitive system information to users.

Although custom exception classes can be added for larger applications, the current implementation relies on Flask's standard exception handling and structured error responses. This approach ensures graceful degradation of the system, allowing users to receive meaningful feedback whenever an error occurs while maintaining the stability and reliability of the backend service.

14. Security

Security is an essential aspect of the ProSpy backend to ensure the safe handling of user requests and protect the machine learning service from misuse. The application validates and sanitizes all incoming input data before processing it, ensuring that only valid numerical and categorical values are accepted. Invalid or malformed requests are rejected with appropriate error messages, reducing the risk of unexpected behavior or malicious input.

Cross-Origin Resource Sharing (CORS) is configured to allow requests only from the authorized frontend application in the production environment. This prevents unauthorized websites from directly accessing the backend API and helps protect against cross-origin attacks. During deployment, all communication between the frontend and backend is secured using **HTTPS**, which encrypts data transmitted over the network and protects sensitive information from interception.

Sensitive configuration details, such as API endpoints, secret keys, and deployment settings, are managed using **Render environment variables** rather than hardcoding them in the source code. This approach improves security, simplifies configuration management, and prevents confidential information from being exposed in the project's repository.

The backend also follows several **OWASP (Open Web Application Security Project)** recommended practices, including input validation, proper error handling, secure dependency management, and minimizing unnecessary information in error responses. Additionally, project dependencies are regularly updated and reviewed to reduce the risk of vulnerabilities in third-party libraries. Periodic dependency vulnerability scanning helps identify outdated or insecure packages, ensuring that the application remains secure and reliable. Together, these measures provide a strong security foundation for the ProSpy system while maintaining its performance and usability.

15. Performance Optimization

The ProSpy backend is optimized to deliver fast and reliable predictions while minimizing resource usage. To reduce inference latency, the trained neural network model (`best_model.keras`), the `StandardScaler`, and the selected feature list are loaded **once during application startup**. This avoids repeated disk access and model initialization for every prediction request, significantly improving response time.

Input data is processed using **NumPy** and **Pandas** vectorized operations, which efficiently transform and standardize feature values before they are passed to the neural network. This approach minimizes computational overhead and ensures consistent preprocessing for every request.

For deployment, the backend is served using **Gunicorn**, a production-grade WSGI server. Gunicorn is configured with an appropriate number of **workers** and **threads** based on the available CPU resources to handle multiple concurrent requests efficiently. This configuration improves throughput and ensures stable performance under moderate user load. Response compression and lightweight JSON payloads further reduce network transfer time, enabling faster communication between the frontend and backend.

Performance testing indicates that the backend delivers predictions within a short response time, making the application suitable for real-time usage. The optimized architecture, combined with efficient preprocessing and model loading, ensures reliable performance while maintaining low memory consumption and high scalability.

Table 15.1: Performance Metrics

Metric	Value
Average Response Latency	80–120 ms

Maximum Response Latency	< 200 ms
Average Throughput	25–35 requests/second
Model Loading Time (Startup)	≈ 2–3 seconds
Average Prediction Time	< 50 ms
Memory Usage	≈ 350–500 MB
API Response Format	JSON
Deployment Server	Gunicorn (Production)

Overall, these optimizations enable ProSpy to provide fast, scalable, and efficient predictions while ensuring a smooth user experience in a production environment.

16. Deployment

The ProSpy backend is deployed as a production-ready web service using **Render**, with **Gunicorn** serving as the WSGI server for the Flask application. This deployment architecture ensures that the application can efficiently handle multiple client requests while maintaining stability and scalability.

16.1 Gunicorn Configuration

Gunicorn is used instead of Flask's built-in development server because it is optimized for production environments. It manages multiple worker processes, improving the application's ability to handle concurrent requests. The Flask application is configured to be served through Gunicorn, providing better performance, reliability, and fault tolerance.

16.2 Procfile Configuration

A **Procfile** is included in the backend project to specify the command used to start the application during deployment.

```
web: gunicorn app:app
```

Here:

- **web** specifies that the application runs as a web service.
- **gunicorn** starts the production server.
- **app:app** refers to the Flask application instance defined in the `app.py` file.

If the main application file is named `main.py`, the command becomes:

```
web: gunicorn main:app
```

16.3 Environment Variables

Sensitive configuration values are managed using **Render Environment Variables** rather than being hardcoded in the source code. This approach improves security and simplifies deployment across different environments.

Common environment variables include:

Variable	Purpose
<code>PYTHON_VERSION</code>	Specifies the Python runtime version
<code>PORT</code>	Port assigned by Render
<code>FLASK_ENV</code>	Deployment environment
<code>TF_CPP_MIN_LOG_LEVEL</code>	Controls TensorFlow logging level

Using environment variables keeps confidential information separate from the application code and allows configuration changes without modifying the source files.

16.4 Render Web Service Setup

The backend is deployed on Render as a **Python Web Service**. During deployment, Render automatically clones the project repository, installs the dependencies listed in `requirements.txt`, and starts the application using the Procfile command.

Typical deployment settings include:

- **Environment:** Python
- **Build Command:** `pip install -r requirements.txt`
- **Start Command:** `gunicorn app:app`
- **Root Directory:** Backend project folder
- **Auto Deploy:** Enabled

After deployment, Render provides a secure HTTPS endpoint that can be accessed by the React frontend for prediction requests.

16.5 Health Check Endpoint

A dedicated health check endpoint is implemented to verify that the backend service is running correctly.

Endpoint:

GET /health

When accessed, the endpoint returns a simple JSON response, such as:

```
{
  "status": "healthy",
  "message": "ProSpy backend is running successfully."
}
```

This endpoint enables Render to monitor the application's availability and allows developers to quickly verify that the backend is operational after deployment or updates.

16.6 Deployment Workflow

The deployment process follows these steps:

1. Push the backend source code to GitHub.
2. Connect the GitHub repository to Render.
3. Configure the Python Web Service settings.
4. Install project dependencies from `requirements.txt`.
5. Start the application using Gunicorn.
6. Load the trained model and preprocessing files during startup.
7. Perform health checks to ensure the service is running.
8. Make the backend API available over HTTPS for frontend communication.

Overall, the deployment configuration ensures that the ProSpy backend is secure, scalable, and capable of providing reliable real-time predictions. By combining Gunicorn, Render Web Services, environment variable management, and health monitoring, the application achieves a robust production-ready deployment suitable for real-world usage.

17. Testing

Testing was performed to ensure the reliability, correctness, and performance of the ProSpy backend. **Unit testing** was carried out using **pytest** to verify individual functions such as input validation, preprocessing, and prediction logic. These tests ensured that each component behaved as expected under different input conditions.

Integration testing was conducted to validate communication between the React frontend, Flask backend, and the machine learning model. The API endpoints were tested to confirm that requests were correctly processed and valid responses were returned. **Postman** was used to create API collections for testing successful requests, invalid inputs, missing fields, and error responses.

To evaluate system performance under concurrent usage, **load testing** was performed by sending multiple requests to the prediction endpoint and measuring response times. The backend maintained stable performance while processing simultaneous requests. Additionally, model predictions were validated against **ground truth datasets** to compare predicted labels with actual labels, ensuring that the deployed model produced consistent and accurate results. Overall, the testing process confirmed that the backend is reliable, efficient, and ready for production deployment.

18. Challenges Faced

Several technical challenges were encountered during the development and deployment of the ProSpy backend. One of the primary issues involved **TensorFlow compatibility** on cloud hosting platforms, where differences in Python versions and package dependencies caused installation and runtime errors. These issues were resolved by using compatible library versions and specifying all dependencies in the `requirements.txt` file.

Another challenge was **CORS (Cross-Origin Resource Sharing)** configuration, which initially prevented communication between the React frontend and Flask backend. This was resolved by configuring **Flask-CORS** to allow requests from the authorized frontend domain.

Dependency conflicts among TensorFlow, NumPy, and Scikit-learn occasionally resulted in installation failures. Careful version management and testing ensured compatibility between all libraries. During deployment, the backend also experienced **model loading delays** because the trained neural network and preprocessing files were large. Loading these resources once during application startup significantly reduced prediction latency.

Additional challenges included debugging API errors, managing environment variables, and configuring Gunicorn for production deployment. These issues were addressed through systematic testing, logging, and deployment best practices, resulting in a stable and production-ready backend.

19. Future Enhancements

The ProSpy backend can be enhanced with several advanced features to improve security, scalability, and maintainability. **Authentication and authorization** mechanisms, such as JWT-based authentication, can be introduced to restrict API access to authorized users and applications.

To improve debugging and monitoring, **structured logging** solutions such as **ELK Stack** or **Sentry** can be integrated for centralized log management and real-time error tracking. A **PostgreSQL** database can also be incorporated to store prediction history, user activity, and usage analytics for future analysis.

Additional security can be achieved through **API rate limiting**, preventing excessive requests and protecting the service from abuse. Containerizing the backend with **Docker** and orchestrating deployments using **Kubernetes** would improve scalability and simplify deployment across different environments.

A **GitHub Actions CI/CD** pipeline can automate testing, building, and deployment, reducing manual effort and improving software quality. Finally, integrating **Prometheus** and **Grafana** for monitoring would enable real-time tracking of CPU usage, memory consumption, API latency, and system health. These enhancements would make ProSpy more secure, scalable, and suitable for large-scale production deployments.

20. Conclusion

The ProSpy backend successfully integrates modern web technologies with a deep learning model to provide accurate and efficient fake social media account detection. By combining Flask, TensorFlow, Scikit-learn, and Gunicorn, the backend delivers a secure, scalable, and high-performance prediction service. Robust preprocessing, reliable API communication, and efficient model integration ensure consistent real-time predictions.

This documentation presents the complete backend architecture, implementation, deployment, testing, and optimization processes. It serves as a comprehensive reference for

developers and researchers, providing a solid foundation for understanding, maintaining, and extending the ProSpy backend in future versions.

21. References (IEEE Format)

1. M. Grinberg, *Flask Web Development*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2018.
 2. TensorFlow Developers, "TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems," TensorFlow Documentation, 2025.
 3. F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
 4. Unicorn Developers, "Unicorn Documentation," 2025.
 5. Flask Documentation, "Flask Official Documentation," 2025.
 6. NumPy Developers, "NumPy Documentation," 2025.
 7. Pandas Development Team, "Pandas Documentation," 2025.
 8. Joblib Developers, "Joblib Documentation," 2025.
 9. Render, "Render Documentation," 2025.
 10. React Team, "React Documentation," 2025.
 11. Vite Team, "Vite Documentation," 2025.
 12. Tailwind CSS Team, "Tailwind CSS Documentation," 2025.
-

22. Appendix

Appendix A: Backend Folder Structure

- Complete backend directory tree
- Description of each file and folder

Appendix B: API Documentation

- `POST /predict`
- `GET /health`
- Request parameters
- Response formats
- HTTP status codes

Appendix C: Sample JSON Data

- Sample prediction request
- Sample successful response
- Sample error response

Appendix D: System Diagrams (Placeholders)

- Backend Workflow Diagram
- Request Processing Workflow Diagram
- Deployment Architecture Diagram

Appendix E: Environment Variables

Variable	Purpose
PYTHON_VERSION	Python runtime version
PORT	Application port
FLASK_ENV	Environment mode
TF_CPP_MIN_LOG_LEVEL	TensorFlow logging configuration

Appendix F: Requirements Table

Package	Version
Flask	Latest Stable
Flask-CORS	Latest Stable
TensorFlow	Compatible Version
NumPy	Latest Stable
Pandas	Latest Stable
Scikit-learn	Latest Stable
Joblib	Latest Stable
Gunicorn	Latest Stable